Formalizing Physics in Lean 4

Michał Pacholski

Software Engineer, Google Warsaw
Faculty of Physics, University of Warsaw (FUW)

30 October 2026

About Me: From Condensed Matter to Software & Back

Academic & Industry Trajectory

- **University of Warsaw (FUW):** BSc & MSc in Physics.
- **Leiden University:** PhD in Theoretical Condensed Matter Physics.
- **MPI-PKS Dresden:** Postdoctoral Researcher in Condensed Matter Theory.
- **Google Warsaw (2024–present):** Software Engineer.

Exploring Lean 4

- Around 2024, I began exploring **Lean 4** as a personal passion project.
- It sits right at the intersection of my two worlds: theoretical physics and modern software engineering.

Outline of Today's Seminar

1. **Why Learn Lean 4?**
   Verification in the age of AI and importing mathematics like a software codebase.
2. **Fundamentals of Functional Programming**
   Pure expressions, immutability, curried functions, and higher-order operators.
3. **Basic Lean 4 Syntax & Data Types**
   `#check`, `#eval`, `def`, products, sums, structures, and inductive types (`Nat`, `List`).
4. **The Curry–Howard Isomorphism**
   Propositions as types, proofs as functional programs, and dependent types.
5. **Proving Theorems in Lean: Term Mode vs. Tactics**
   Direct term construction vs. interactive tactic proofs (`intro`, `rw`, `simp`).
6. **Condensed Matter Case Study: The 1D Tight-Binding Chain**
   Periodic boundary conditions via `Fin N`, Bloch waves via `rootsOfUnity`, and an exact orthonormal eigenbasis.

Why Learn Lean 4? I. Verification in the Age of AI

Informal Derivations Can Be Hallucinated

- LLMs can generate natural-language mathematical arguments and informal derivations that read smoothly to a human.
- Yet the entire argument—or a critical link in the reasoning—can be completely hallucinated.
- Checking an LLM's informal derivation line-by-line is slow, exhausting, and error-prone.

The Solution: Dual Output

- Require the AI to produce **both** an informal explanation *and* formal **Lean 4 code**.
- Lean's kernel automatically verifies that the proof is logically sound.
- **Your job as a physicist:** Check that the Lean `def` and `theorem` statements actually match the informal claim!

```
-- You don't need to read the proof behind "by"; just verify the statement matches the claim:
theorem isEigenvector_planeWave :
    (hamiltonian N μ t) *ᵥ planeWave N u =
    (μ + t * u.val⁻¹ + star t * u.val) • planeWave N u := by ...
```

Why Learn Lean 4? II. Physics as an Importable Codebase

How Software Scales (e.g. at Google)

- Thousands of engineers collaborate on one massive codebase over many years.
- You don't rewrite basic utilities—you `import` existing libraries, and the compiler guarantees compatibility.

How Theoretical Physics Works Today

- Results live in static PDF papers with varying notations and implicit assumptions.
- Every new paper or textbook re-derives the same sums, integrals, and linear algebra from scratch.

Lean's `Mathlib`: A unified codebase of pure mathematics (linear algebra, analysis, algebra) that we can directly `import` to build a shared codebase for physics:

```
import Mathlib.LinearAlgebra.Matrix.Hermitian
import Mathlib.RingTheory.RootsOfUnity.Complex
import Mathlib.Analysis.InnerProductSpace.PiL2
```

Fundamentals of Functional Programming

Imperative (Python, C++, Fortran)

- Sequence of **state mutations** (`x = x + 1`) and in-place side effects.

Functional (Lean 4, Haskell, OCaml)

- Evaluation of **pure, immutable expressions**: `f x` always yields the same result.

Key Functional Concepts in Lean 4:

- **Everything has a type (`x : T`):** Numbers, functions, types, and even proofs.
- **First-class functions (`f : A → B`):** A function mapping inputs of type `A` to outputs of type `B` has type signature `A → B`. Functions are ordinary values and can be passed to higher-order functions (like operators in physics).
- **Currying (`A → B → C`):** Read as `A → (B → C)`—a two-argument function takes an `A` and returns a new function of type `B → C`.

```
-- Anonymous function (lambda) of type ℕ → ℕ:
fun (x : ℕ) ↦ x + 1
-- Higher-order function of type (ℕ → ℕ) → ℕ → ℕ:
def applyTwice (f : ℕ → ℕ) (x : ℕ) : ℕ := f (f x)
```

Basic Lean 4 Syntax: def, #check, and #eval

Interacting with the Lean 4 Compiler:

- `def` binds a name to a value or function with explicit parameter and return types.
- `#check` asks Lean to infer and display the **type signature** of an expression without evaluating it.
- `#eval` executes a computable expression and prints its resulting **value**.

```
-- Define a constant and a curried two-argument function:
def n : ℕ := 5
def add (x : ℕ) (y : ℕ) : ℕ := x + y

-- Inspect types with #check (notice how partial application works!):
#check n          -- n : ℕ
#check add        -- add : ℕ → ℕ → ℕ
#check add 2      -- add 2 : ℕ → ℕ   (partially applied function!)
#check add 2 3    -- add 2 3 : ℕ

-- Compute values with #eval:
#eval add 2 3     -- 5
```

Combining Data I: Product Types and `structure`

Holding Multiple Values Simultaneously:

- **Product Types (`A × B`):** Hold **both** an `A` *and* a `B` as an anonymous pair `(a, b)`. Fields are accessed via `.1` and `.2` (or `.fst` and `.snd`).
- **Structures (`structure`):** Define a custom, named product type with readable field names instead of positional indices.

```
-- Anonymous product type (pair):
def p : ℕ × ℝ := (3, 2.5)
#eval p.1    -- 3
#eval p.2    -- 2.5

-- Named structure:
structure Point2D where
  x : ℝ
  y : ℝ

def pt : Point2D := { x := 1.0, y := -1.0 }
#check pt.x  -- pt.x : ℝ
```

Combining Data II: Sum Types (`A ⊕ B`)

Representing Alternatives (Disjoint Unions):

- **Sum Types (`A ⊕ B` or `Sum A B`):** Hold **either** a value of type `A` (tagged with `Sum.inl a`) *or* a value of type `B` (tagged with `Sum.inr b`).
- **Pattern Matching (`match ... with`):** To use a sum type, we must provide a branch for every possible constructor—the compiler guarantees no case is forgotten!

```
-- Constructing values of a Sum type (ℕ ⊕ ℝ):
def s1 : ℕ ⊕ ℝ := Sum.inl 42
def s2 : ℕ ⊕ ℝ := Sum.inr 3.14

-- Consuming a Sum type via pattern matching:
def toReal (s : ℕ ⊕ ℝ) : ℝ :=
  match s with
  | Sum.inl n ↦ (n : ℝ)
  | Sum.inr r ↦ r
```

Inductive Data Types: `Nat` and `List`

In Lean 4, even numbers and containers are not built-in "magic" primitives—they are defined from scratch as **recursive inductive types** (`inductive`):

1. **Peano Natural Numbers (`Nat` / $\mathbb{N}$)**
   - `zero`: the base number 0.
   - `succ n`: the successor $n + 1$ of an existing `n : Nat`.
   - 2 is syntax sugar for `succ (succ zero)`.
2. **Polymorphic Lists (`List` α)**
   - `nil`: the empty list `[]`.
   - `cons head tail`: an element `head : α` prepended to `tail : List α` (written `head :: tail`).

```
-- How Nat and List are defined in Lean's standard library:
inductive Nat where
  | zero : Nat
  | succ (n : Nat) : Nat

inductive List (α : Type) where
  | nil  : List α
  | cons (head : α) (tail : List α) : List α
```

Pattern Matching & Structural Recursion

Functions on inductive types are defined by **pattern matching** on each constructor. Lean ensures termination by checking that recursive calls only happen on smaller subterms:

```
-- Addition on Nat by recursion on the second argument:
def myAdd (m n : Nat) : Nat :=
  match n with
  | Nat.zero   ↦ m
  | Nat.succ k ↦ Nat.succ (myAdd m k)

-- Length of a polymorphic List (using [] for nil and x :: xs for cons):
def myLength (xs : List α) : Nat :=
  match xs with
  | []      ↦ 0
  | _ :: tl ↦ 1 + myLength tl

#eval myAdd 3 2              -- 5
#eval myLength [10, 20, 30] -- 3
```

The Curry–Howard Isomorphism: Propositions as Types

In Lean 4, there is no boundary between **programming** and **proving**:

- A mathematical **proposition** P is simply a **type** (`P : Prop`).
- A **proof** of P is a **program (value)** p of type P (`p : P`).
- Checking if a proof is valid is literally **type-checking** the program!

Functional Programming	**Lean 4 Syntax**	**Logic (Mathematics)**
Type	`P : Prop`	Proposition P
Value / Term of type P	`h : P`	Proof of P
Empty / Unit type	`Empty/Unit`	
Function type	`P` $\to$ `Q`	
Product type	`A` $\times$ `B`	
Sum type	`A` $\oplus$ `B`	

The Curry–Howard Isomorphism: Propositions as Types

In Lean 4, there is no boundary between **programming** and **proving**:

- A mathematical **proposition** P is simply a **type** (`P : Prop`).
- A **proof** of P is a **program (value)** p of type P (`p : P`).
- Checking if a proof is valid is literally **type-checking** the program!

Functional Programming	**Lean 4 Syntax**	**Logic (Mathematics)**
Type	`P : Prop`	Proposition P
Value / Term of type P	`h : P`	Proof of P
Empty / Unit type	`Empty`/`Unit` (`False`/`True`)	Falsehood / Truth
Function type	`P` $\to$ `Q`	
Product type	`A` $\times$ `B`	
Sum type	`A` $\oplus$ `B`	

The Curry–Howard Isomorphism: Propositions as Types

In Lean 4, there is no boundary between **programming** and **proving**:

- A mathematical **proposition** P is simply a **type** (`P : Prop`).
- A **proof** of P is a **program (value)** p of type P (`p : P`).
- Checking if a proof is valid is literally **type-checking** the program!

Functional Programming	**Lean 4 Syntax**	**Logic (Mathematics)**
Type	`P : Prop`	Proposition P
Value / Term of type P	`h : P`	Proof of P
Empty / Unit type	`Empty`/`Unit` (`False`/`True`)	Falsehood / Truth
Function type	`P` $\to$ `Q`	Implication ($P \to Q$)
Product type	`A` $\times$ `B`	
Sum type	`A` $\oplus$ `B`	

The Curry–Howard Isomorphism: Propositions as Types

In Lean 4, there is no boundary between **programming** and **proving**:

- A mathematical **proposition** P is simply a **type** (`P : Prop`).
- A **proof** of P is a **program (value)** p of type P (`p : P`).
- Checking if a proof is valid is literally **type-checking** the program!

Functional Programming	**Lean 4 Syntax**	**Logic (Mathematics)**
Type	`P : Prop`	Proposition P
Value / Term of type P	`h : P`	Proof of P
Empty / Unit type	`Empty`/`Unit` (`False`/`True`)	Falsehood / Truth
Function type	`P` $\to$ `Q`	Implication ($P \to Q$)
Product type	`A` $\times$ `B` (`P` $\wedge$ `Q`)	Conjunction ($P \wedge Q$)
Sum type	`A` $\oplus$ `B`	

The Curry–Howard Isomorphism: Propositions as Types

In Lean 4, there is no boundary between **programming** and **proving**:

- A mathematical **proposition** P is simply a **type** (`P : Prop`).
- A **proof** of P is a **program (value)** p of type P (`p : P`).
- Checking if a proof is valid is literally **type-checking** the program!

Functional Programming	**Lean 4 Syntax**	**Logic (Mathematics)**
Type	`P : Prop`	Proposition P
Value / Term of type P	`h : P`	Proof of P
Empty / Unit type	`Empty`/`Unit` (`False`/`True`)	Falsehood / Truth
Function type	`P` $\to$ `Q`	Implication ($P \to Q$)
Product type	`A` $\times$ `B` (`P` $\wedge$ `Q`)	Conjunction ($P \wedge Q$)
Sum type	`A` $\oplus$ `B` (`P` $\vee$ `Q`)	Disjunction ($P \vee Q$)

Dependent Types: Quantifiers ($\forall$, $\exists$) and Subtypes

In Lean 4, **types can depend on values**: a family of propositions $P(x)$ is simply a function `P : α → Prop` mapping each value `x : α` to a proposition `P x`.

- **Universal Quantifier ($\forall x : \alpha, P(x)$) $\longleftrightarrow$ Dependent Function (`(x : α) → P x`):**
  A function that takes any input `x : α` and returns a proof of `P x` for that specific `x`.
- **Existential Quantifier ($\exists x : \alpha, P(x)$) $\longleftrightarrow$ Dependent Pair (`⟨x, h⟩`):**
  A structure packing a concrete witness `x : α` together with a proof `h : P x`.
- **Data + Proofs in Physics (e.g. `Fin N`):**
  A site on an N-site ring has type `Fin N`: a pair bundling a number `val : ℕ` with a proof `isLt : val < N`. Out-of-bounds indices cannot even be constructed!

```
-- Universal quantifier (∀ n : ℕ, n = n) is a dependent function:
#check ∀ (n : ℕ), n = n      -- ∀ (n : ℕ), n = n : Prop

-- Fin N bundles a natural number with a proof that it is strictly less than N:
structure Fin (N : ℕ) where
  val  : ℕ
  isLt : val < N
```

Proving by Direct Term Construction (Term Mode)

Because a proof of P is just a value of type P, we can prove theorems directly by writing a functional program of that type (**Term Mode**):

- `theorem name (args) : Statement := proof_term` is identical to `def`, except its return type lives in `Prop`.
- **Implication ($P \to Q$):** Proven by a function `fun h => ...` mapping a proof `h : P` to a proof of `Q`.
- **Conjunction / Disjunction:** Proven using pair and sum constructors (`And.intro`, `Or.inl`).

```
-- 1. Proving P → P using the identity function:
theorem id_proof (P : Prop) : P → P :=
  fun (h : P) ↦ h

-- 2. Proving P ∧ Q → Q ∧ P by swapping a pair:
theorem and_swap (P Q : Prop) : P ∧ Q → Q ∧ P :=
  fun (h : P ∧ Q) ↦ And.intro h.right h.left

-- 3. Proving P → P ∨ Q by injecting into the left branch:
theorem or_left (P Q : Prop) : P → P ∨ Q :=
  fun (h : P) ↦ Or.inl h
```

Interactive Tactic Mode and the Lean Infoview

For complex theorems, writing raw proof terms by hand becomes unwieldy. Instead, we enter **Tactic Mode** using by:

- **Tactics** are metaprograms that build the underlying proof term step by step.
- At every cursor position, the **Lean Infoview** displays the current **Proof State**: local hypotheses at the top, and the remaining goal after the turnstile (⊢).

Tactic Script (`by`):

```
theorem and_swap (P Q : Prop) :
    P ∧ Q → Q ∧ P := by
  intro h
  constructor
  · exact h.right
  · exact h.left
```

Lean Infoview (after `intro h`):

```
P Q : Prop
h : P ∧ Q
⊢ Q ∧ P

-- `constructor` splits ⊢ Q ∧ P
-- into two subgoals: ⊢ Q and ⊢ P
```

Core Tactics: `rw`, `simp`, `ring`, and `induction`

- `rw [h]` (**Rewrite**): Substitutes equals for equals in the goal using equation `h` (use `rw [← h]` to rewrite right-to-left).
- `simp` (**Simplifier**): Automatically applies tagged simplification lemmas (`@[simp]`) to reduce the goal.
- `ring`: Automatically proves any polynomial identity in commutative (semi)rings.
- `induction n with`: Performs structural induction on an inductive type like $\mathbb{N}$.

```
-- 1. Rewriting with a hypothesis h : a = b
example (a b c : ℝ) (h : a = b) : a + c = b + c := by
  rw [h]

-- 2. Automatic ring algebra (no manual expansion needed!)
example (x y : ℝ) : (x + y)^2 = x^2 + 2*x*y + y^2 := by
  ring

-- 3. Proving 0 + n = n by mathematical induction on n
theorem zero_add (n : ℕ) : 0 + n = n := by
  induction n with
  | zero      ↦ rfl
  | succ k ih ↦ rw [Nat.add_succ, ih]
```

1D Tight-Binding Chain: Hamiltonian & Hermiticity

We model an N-site periodic ring (`Fin N`) with onsite chemical potential $\mu \in \mathbb{R}$ and nearest-neighbor hopping $t \in \mathbb{C}$ as an $N \times N$ matrix $H_{ij} = \mu\delta_{ij} + t\delta_{i,j+1} + t^*\delta_{j,i+1}$:

```
variable (N : ℕ) [NeZero N] (μ : ℝ) (t : ℂ)

def onsite (i j : Fin N) := if i = j then (μ : ℂ) else 0

def hopping_forward (i j : Fin N) := if i = j + 1 then t else 0

def hamiltonian_fun (i j : Fin N) :=
  onsite N μ i j + hopping_forward N t i j + star (hopping_forward N t j i)

def hamiltonian := Matrix.of (hamiltonian_fun N μ t)

theorem isHermitian_hamiltonian : (hamiltonian N μ t).IsHermitian := by
  ext i j
  unfold hamiltonian hamiltonian_fun
  simp only [Matrix.conjTranspose_apply, Matrix.of_apply, star_add,
    star_star, onsite_conjTranspose_eq_self, add_comm, add_assoc]
```

Bloch Eigenstate Theorem: Step-by-Step Derivation

Claim: For $\psi_u(j) = \frac{u^j}{\sqrt{N}}$ with $u^N = 1$, we have $H\psi_u = (\mu + tu^{-1} + t^* u)\psi_u$.

1. **Expand the matrix-vector product at site $i \in \{0, \ldots, N-1\}$:**

$$(H\psi_u)_i = \sum_{j=0}^{N-1} \left(\mu\, \delta_{i,j} + t\, \delta_{i,\,(j+1) \bmod N} + t^*\, \delta_{j,\,(i+1) \bmod N} \right) \frac{u^j}{\sqrt{N}}$$

2. **Evaluate the Kronecker deltas:** Rewriting $i \equiv j + 1 \pmod{N} \iff (i-1) \bmod N = j$ collapses the sum to:

$$(H\psi_u)_i = \mu\, \frac{u^i}{\sqrt{N}} + t\, \frac{u^{(i-1) \bmod N}}{\sqrt{N}} + t^*\, \frac{u^{(i+1) \bmod N}}{\sqrt{N}}$$

3. **Apply periodicity ($u^m = u^{m \bmod N}$ since $u^N = 1$):** In the third term, $u^{(i+1) \bmod N} = u^{i+1}$. In the second term, subtraction modulo N is defined by shifting by N: $(i-1) \bmod N = (N - 1 + i) \bmod N$.

4. **Check boundary cases on N and factor:**
   - For $N = 1$: $u = 1$ and $i = 0$, so both sides immediately equal $\mu + t + t^*$.
   - For $N \geq 2$: $u^{(N-1+i) \bmod N} = u^{N-1+i} = u^{N-1} u^i = u^{-1} u^i$, giving $(\mu + tu^{-1} + t^* u)\frac{u^i}{\sqrt{N}}$.

Bloch Eigenstate Proof in Lean 4

```
theorem isEigenvector_planeWave :
    (hamiltonian N μ t) *ᵥ planeWave N u =
    (μ + t * u.val⁻¹ + star t * u.val) • planeWave N u := by
  ext x'; rw [Matrix.mulVec_apply]                          -- 1. Entrywise at site x'
  unfold hamiltonian hamiltonian_fun planeWave planeWaveFun dotProduct hopping_forward onsite
  simp only [Matrix.row_apply, Matrix.of_apply, add_mul, apply_ite star, star_zero,
    ite_mul, zero_mul, Finset.sum_add_distrib, Finset.sum_ite_eq, Finset.sum_ite_eq',
    Finset.sum_ite_add_eq_sum_ite_sub, Finset.mem_univ, ite_true, Pi.smul_apply,
    smul_eq_mul, Fin.val_sub, Fin.val_add, Fin.val_one']  -- 2. Collapse Kronecker δs
  have pow_mod : ∀ m : ℕ, u ^ m = u ^ (m % N : ℕ) := by   -- 3. u^m depends only on m mod N
    intro m; obtain ⟨uv, hu⟩ := u
    simp only [SubmonoidClass.mk_pow, Subtype.mk.injEq]; exact pow_eq_pow_mod m hu
  simp only [SubmonoidClass.coe_pow, Units.val_pow_eq_pow_val, ← pow_mod, RCLike.star_def,
    Nat.add_mod_mod, Units.val_inv_eq_inv_val]
  rcases N with _ | N | N                                   -- 4. Split N = 0, N = 1, N ≥ 2
  · exact absurd rfl (NeZero.ne 0)
  · norm_num; have hu : u = 1 := by simpa using u.property
    simp [hu]
  · norm_num; norm_cast; simp [inv_u]; ring
```

Orthonormality & Complete Eigenbasis

Finally, we prove that the Bloch waves are orthonormal via the geometric series formula (`geom_sum_eq`) for $u'/u \neq 1$, and promote them to a complete `OrthonormalBasis` of $\mathbb{C}^N$:

```
theorem orthonormal_planeWaves : Orthonormal ℂ (planeWave N) := by
  constructor
  · intro u            -- Normalization: ∑ x, ‖planeWaveFun N u x‖ ^ 2 = 1
    ...
  · intro u u' h_ne    -- Orthogonality (u ≠ u'): geometric sum vanishes
    ...
    simp [Fin.sum_univ_eq_sum_range, geom_sum_eq h_ne']
    simp [div_pow]; norm_cast; simp [u_pow]

-- Since |rootsOfUnity N ℂ| = N = dim(ℂ ^ N), orthonormality implies completeness:
noncomputable def planeWaveBasis :
    Module.Basis (rootsOfUnity N ℂ) ℂ (EuclideanSpace ℂ (Fin N)) := by
  apply basisOfOrthonormalOfCardEqFinrank (orthonormal_planeWaves N)
  rw [Fintype.card_eq_nat_card, Complex.card_rootsOfUnity]; simp

noncomputable def planeWaveOrthonormalBasis :
    OrthonormalBasis (rootsOfUnity N ℂ) ℂ (EuclideanSpace ℂ (Fin N)) :=
  Basis.toOrthonormalBasis (planeWaveBasis N) (orthonormal_planeWavesBasis N)
```

Summary & Resources for Getting Started

Key Takeaways:

- **Curry–Howard Isomorphism:** Propositions are types, proofs are programs, and proof verification is just type-checking.
- **Expressive Type System:** Dependent types (`Fin N`, `rootsOfUnity N ℂ`) encode physical constraints and symmetries directly in definitions.
- **Interactive Proving:** Tactics (`rw`, `simp`, `ring`) automate routine algebra while the Infoview tracks every logical assumption and goal.

Where to Start:

- **Code from this Seminar:** `github.com/mpacholski/physics_in_lean`
- **The Natural Number Game:** Interactive browser puzzle game teaching tactics from scratch (`adam.math.hhu.de`).
- **Official Textbooks:** *Theorem Proving in Lean 4*, *Mathematics in Lean*, & *Functional Programming in Lean* (`lean-lang.org/documentation`).
- **Mathlib4 & PhysLean (HepLean):** Community libraries formalizing mathematics and physics (`leanprover-community.github.io`).
- **Lean Zulip Chat:** Active, welcoming forum for questions and collaboration (`leanprover.zulipchat.com`).